
Oliver Krauss

Generics & Collections

ALD | MTD.ba VZ SS26

Hagenberg 08.04.2026

HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Recap: Where We Left Off

Before we start -- let's connect to your last exercise.

You previously implemented **Selectable** and **Transformable** interfaces, and sorted shapes using a lambda: `shapes.sort(Comparator.comparing(Shape::getArea))`. You saw the notation **Comparator<T>** and **<T>** was mentioned on some slides.

Recap: Where We Left Off

Before we start -- let's connect to your last exercise.

You previously implemented **Selectable** and **Transformable** interfaces, and sorted shapes using a lambda: `shapes.sort(Comparator.comparing(Shape::getArea))`. You saw the notation **Comparator<T>** and **<T>** was mentioned on some slides.

Previously we learned:

- **Interfaces**: **Comparable<T>**, **Comparator<T>** -- the **<T>** was there all along
- **Lambdas**: concise implementations of functional interfaces
- **Data Structures**: many data structures, but no generic ones.

Today: we unlock generics, then use them to master the Java Collections Framework.

The Problem: A Box Without Generics

Suppose we want a reusable container that holds one value:

```
1  public class Box {  
2      private Object value;  
3  
4      public void set(Object value) { this.value = value; }  
5      public Object get() { return this.value; }  
6  }
```

This compiles -- but causes problems at runtime:

```
1  Box box = new Box();  
2  box.set("Hello");  
3  
4  // Compiler allows this -- Object can hold anything:  
5  Integer n = (Integer) box.get(); // COMPILES OK -- but explodes at runtime:  
6  // ClassCastException! It's a String, not an Integer.
```

Introducing Type Parameters

Generics let us write code that works with **any type** while preserving type safety:

```
1    public class Box<T> {           // T is a type parameter (placeholder)
2        private T value;
3
4        public void set(T value) { this.value = value; }
5        public T get() { return this.value; }
6    }
```

Key idea

T is a **placeholder for a real type**. When you write `Box<String>`, every **T** in the class becomes **String**. The compiler checks everything **at compile time**.

(Convention: **T** for general type, **E** for element, **K/V** for key/value. Any valid identifier works -- the letter is just a hint to the reader.)

Type Parameter in Action

Now the compiler catches mistakes:

```
1  Box<String> strBox = new Box<>();
2  strBox.set("Hello");
3  String s = strBox.get();          // no cast needed -- type is known!
4
5  Box<Integer> intBox = new Box<>();
6  intBox.set(42);
7  intBox.set("oops");              // COMPILER ERROR: expected Integer
```

Compile-time safety

The type mismatch is rejected before running the program.

Type Inference with <>

Java can infer the type argument -- you don't always need to repeat it:

```
1 // Old way (Java 6): repeat the type on both sides
2 Box<String> box1 = new Box<String>();
3
4 // Diamond operator (Java 7+): compiler infers from left side
5 Box<String> box2 = new Box<>();    // <> = "diamond"
6
7 // Works for any generic type:
8 ArrayList<Shape> shapes = new ArrayList<>();    // infers ArrayList<Shape>
9 HashMap<String, Shape> map = new HashMap<>();    // infers HashMap<String, Shape>
```

Diamond operator <>

The compiler infers the type argument from the declared type on the left side. This is standard style since Java 7 -- always prefer **new ArrayList<>()** over **new ArrayList<Shape>()**.

Generic Classes: Our Shape Container

Let's build a typed drawing container for the shapes running example:

```
1  public class Drawing<T> {
2      private T[] shapes;
3      private int count;
4
5      public Drawing(int capacity) {
6          shapes = (T[]) new Object[capacity];
7      }
8
9      public void add(T shape) {
10         shapes[count++] = shape;
11     }
12
13     public T get(int index) {
14         return shapes[index];
15     }
16 }
```


Java quirk: generic array creation

Why this cast appears

Java cannot directly create a generic array (`new T[capacity]` does not compile), so examples often use:

```
1  @SuppressWarnings("unchecked")
2  shapes = (T[]) new Object[capacity];
```

Practical rule

In real code, prefer `ArrayList<T>` internally. This array version is mainly educational, showing how `T` acts as a compile-time placeholder. (Appendix covers the full explanation.)

Bounded Type Parameters

Sometimes we need **T** to be a **specific kind** of type.

```
1  // Without bound: T could be anything -- cannot call shape methods
2  public class Drawing<T> {
3      public double totalArea(T shape) {
4          return shape.getArea(); // ERROR: T has no getArea() method!
5      }
6  }
```

The problem

Without a bound, the compiler only knows “this is some type T” and cannot assume Shape methods exist.

Upper bound solution: <T extends Shape>

```
1  // With upper bound: T must be Shape or a subtype of Shape
2  public class Drawing<T extends Shape> {
3      public double totalArea() {
4          double total = 0;
5          for (int i = 0; i < count; i++) {
6              total += shapes[i].getArea(); // OK: T is guaranteed a Shape
7          }
8          return total;
9      }
10 }
```

<T extends Shape> means

“T can be any type, **as long as it is a Shape** (or a subclass of Shape).” This lets you call Shape methods on T safely.

Generic Methods

A method can have its own type parameter, independent of the class:

```
1  public class Utils {
2      // Generic method: <T> declared before return type
3      public static <T> T first(T[] array) {
4          if (array.length == 0) return null;
5          return array[0];
6      }
7
8      public static <T> void swap(T[] array, int i, int j) {
9          T temp = array[i];
10         array[i] = array[j];
11         array[j] = temp;
12     }
13 }
```

The **<T>** before the return type declares the type parameter for **this method only**.

Generic Methods: Type Inference in Calls

```
1  String[] names = {"Alice", "Bob", "Carol"};
2  String s = Utils.first(names);    // T inferred as String
3
4  Integer[] nums = {10, 20, 30};
5  Utils.swap(nums, 0, 2);          // T inferred as Integer
```

Inference

You usually do not write explicit type arguments for method calls; the compiler infers **T** from parameters and expected return type.

Interactive Break: Predict the Type Error

Which of these snippets causes a compile error? Where exactly?

A)

```
1 Box<String> box = new Box<>();  
2 box.set(123);  
3 String s = box.get();
```

B)

```
1 Box<Integer> box = new Box<>();  
2 box.set(42);  
3 String s = (String) box.get();
```

C)

```
1 public class Drawing<T extends Shape> {  
2     public void add(T s) { /* ... */ }  
3 }  
4  
5 Drawing<Circle> d = new Drawing<>();  
6 d.add(new Circle(0, 0, 10, Color.RED));
```

D)

```
1 Drawing<String> d = new Drawing<>();
```

Interactive Break: Answers

A) Compile error on `box.set(123)`

`Box<String>` only accepts Strings. Passing `int` (auto-boxed to `Integer`) fails at compile time.

B) Runtime error on `(String) box.get()`

Casting to `String` will not work -- `Integer` and `String` are unrelated types with no common supertype other than `Object`. The runtime will throw a `ClassCastException`.

C) Compiles and runs fine

`Circle` extends `Shape` satisfies the `<T extends Shape>` bound. Adding a `Circle` to a `Drawing<Circle>` is perfectly legal.

D) Compile error

`String` does not extend `Shape`. The bound `T extends Shape` is violated.

Where Generics Show Up in Java

Everywhere in the Java Collections Framework:

Type	Type parameter	Meaning
<code>List<T></code>	element type	a list of T's
<code>Comparable<T></code>	comparison target	compares to T
<code>Comparator<T></code>	compared type	orders T's
<code>Optional<T></code>	value type	may contain a T
<code>Predicate<T></code>	input type	tests a T, returns boolean
<code>Function<T,R></code>	input + output	maps T to R

Wildcard <?>: any element type

Use <?> when you only need to **read** elements and do not care about their concrete type:

```
1  // "I accept a list of anything"
2  public static void printAll(List<?> list) {
3      for (Object item : list) {
4          System.out.println(item);
5      }
6  }
```

Meaning

<?> = unbounded wildcard. Accepts a list of any element type, but you cannot safely add specific values.

Wildcard `<? extends Shape>`: upper bound

Use an upper bound when callers may pass **Shape** or any subtype, and you need Shape behavior:

```
1  // "I accept a list of Shape or any subtype"
2  public static double sumAreas(List<? extends Shape> shapes) {
3      double total = 0;
4      for (Shape s : shapes) {
5          total += s.getArea();    // safe: we know it's at least a Shape
6      }
7      return total;
8  }
```

- `<? extends Shape>`: upper-bounded wildcard; readable as Shape
- Typically read-only for API consumers (cannot add arbitrary Shape values)
- Wildcards appear mainly in **library/API code** -- you will encounter them, but rarely write them yourself

Recap -- Generics

- **Generics** let you write type-safe, reusable code with a **type placeholder** `<T>`
- Without generics: `Object` + casts → runtime `ClassCastException`
- With generics: errors are caught **at compile time**
- **Generic class**: `class Box<T>` -- `T` is replaced when instantiated
- **Bounded type parameter**: `<T extends Shape>` -- restricts which types are allowed
- **Generic method**: `public static <T> T first(T[] a)` -- own type param per method
- **Diamond operator** `<>`: shorthand, compiler infers the type argument
- **Wildcards** `<?>`, `<? extends T>`: useful for API method parameters (especially read-only inputs)

Any questions before we put generics to work with the Collections Framework?

From manual arrays to collection APIs

You already know the underlying structures. The issue in day-to-day code is the **manual bookkeeping** arrays force:

```
1  Shape[] shapes = new Shape[10];    // size fixed at creation time!
2  int count = 0;
3
4  shapes[count++] = new Circle(...);
5  shapes[count++] = new Rectangle(...);
6
7  // Remove the first shape -- manual shifting required:
8  for (int i = 0; i < count - 1; i++) {
9      shapes[i] = shapes[i + 1];
10 }
11 shapes[--count] = null;              // avoid memory leak
```

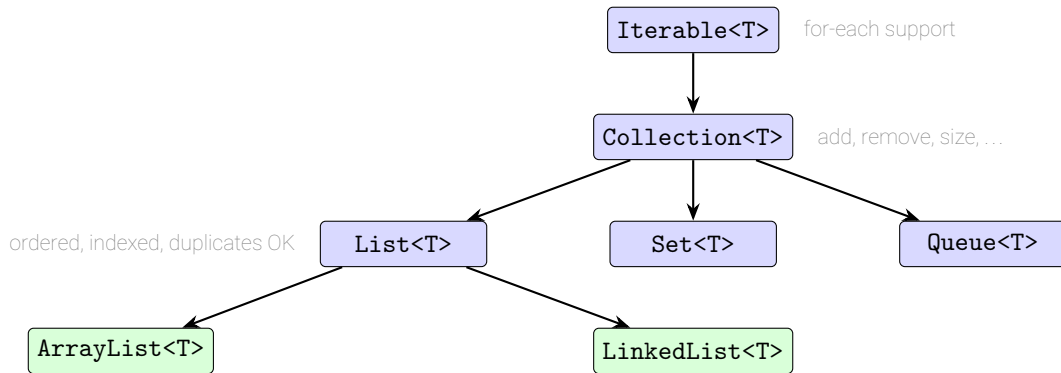
Why move to Collections now?

Arrays are fine, but API-level work wants more

- **Resizing logic**: capacity and bounds management by hand
- **Remove/update logic**: manual shifts and index maintenance
- **Boilerplate loops**: contains/filter/find repeated everywhere
- **Readability**: business logic gets mixed with storage mechanics

Collections Framework Overview

Java provides a hierarchy of ready-made, generic data structures:



ArrayList<T>: The Most-Used Collection

ArrayList is a resizable array -- the go-to choice for most situations:

```
1  List<Shape> shapes = new ArrayList<>();    // prefer List<T> on the left!
2
3  // Adding elements
4  shapes.add(new Circle(100, 100, 50, Color.RED));
5  shapes.add(new Rectangle(200, 200, 80, 40, Color.BLUE));
6  shapes.add(0, new Circle(50, 50, 20, Color.GREEN)); // insert at front
7
8  // Reading
9  Shape first = shapes.get(0);                // the Circle at front
10 int count   = shapes.size();                // 3
11 boolean has = shapes.contains(first); // true -- uses equals()
12
13 // Removing
14 shapes.remove(first);                       // by reference -- uses equals()
15 shapes.remove(0);                          // by index
```

ArrayList<T>: Two key habits

Two key habits

1. Declare as **List<Shape>**, not **ArrayList<Shape>** -- keeps options open (“program to the interface”).
2. **remove(object)** relies on **equals()** -- override **equals()** in your classes, or removal may silently fail.

Programming to the Interface

Declare variables and parameters using the **interface type**, not the concrete class:

```
1 // Prefer this:
2 List<Shape> shapes;
3 List<Shape> getShapes() { ... }
4 void render(List<Shape> s) { ... }

vs. // Avoid this:
ArrayList<Shape> shapes;
ArrayList<Shape> getShapes() { ... }
void render(ArrayList<Shape> s) { ... }
```

Why it matters

This works because **ArrayList implements** the **List** interface -- just like a **Circle** is-a **Shape**, an **ArrayList** is-a **List**.

If you later need to switch from **ArrayList** to **LinkedList** (or any other **List** implementation), you only change **one line** (the **new ArrayList<>()**) -- callers are unaffected. This is the same principle as coding against the **Shape** interface rather than **Circle** directly.

ArrayList in Action: Shape Drawing

Replacing the fixed-size array with an **ArrayList**. No fixed capacity, no manual shifting on remove, built-in iteration support. The **ArrayList** handles all bookkeeping.

```
1  public class DrawingApp {
2      private List<Shape> shapes = new ArrayList<>();
3
4      public void addShape(Shape s) {
5          shapes.add(s);
6      }
7      public void removeShape(Shape s) {
8          shapes.remove(s);    // no manual shifting!
9      }
10     public void render(GraphicsContext gc) {
11         for (Shape s : shapes) {    // enhanced for-loop
12             s.draw(gc);
13         }
14     }
15 }
```

The Enhanced For-Loop

Java's enhanced for-loop works on any **Iterable** -- including all Collections:

```
1  // Index-based loop -- use when you need the index:
2  for (int i = 0; i < shapes.size(); i++) {
3      System.out.println(i + ": " + shapes.get(i));
4  }
5  // Enhanced for-loop -- use when you just need each element:
6  for (Shape s : shapes) {
7      s.draw(gc); // (gc is the JavaFX GraphicsContext from DrawingApp)
8  }
```

Use index loop when

- You need the index value
- You are modifying the list while iterating
(use **Iterator** instead -- next slide)
- Parallel iteration over two lists

Use enhanced for-loop when

- You just need each element
- Code is cleaner and intention is clear
- Works on arrays, Lists, Sets, any
Iterable

Iterator<T>: What it is

An **Iterator** is an object that walks through a collection one element at a time.

Core methods

- **hasNext()**: is there another element?
- **next()**: return the next element and advance
- **remove()**: remove the element returned by the last **next()**

When to care

If you need to remove elements while traversing a collection, the iterator API is the safe mechanism.

Wrong: remove inside for-each

Removing elements **during** a for-each loop throws an exception:

```
1  // WRONG -- causes ConcurrentModificationException:
2  for (Shape s : shapes) {
3      if (s.getArea() < 1.0) {
4          shapes.remove(s);    // modifying while iterating!
5      }
6  }
```

Safe removal with `Iterator<T>`

```
1  import java.util.Iterator;
2
3  Iterator<Shape> it = shapes.iterator();
4  while (it.hasNext()) {
5      Shape s = it.next();
6      if (s.getArea() < 1.0) {
7          it.remove();    // safe: Iterator tracks state internally
8      }
9  }
```

Modern Java 8+ preferred approach

```
shapes.removeIf(s -> s.getArea() < 1.0);
```

Same result, one line -- uses a lambda directly. We will use **`removeIf()`** throughout the rest of the course.

LinkedList<T>: Doubly-Linked Under the Hood

LinkedList implements the same **List** interface -- but stores elements in linked nodes:

`null ← [prev|"zero"|next] ↔ [prev|"first"|next] ↔ [prev|"second"|next] → null`

```
1  LinkedList<String> list = new LinkedList<>();
2
3  // Standard List operations (same interface as ArrayList):
4  list.add("first");
5  list.add("second");
6
7  // Extra operations (front/back access):
8  list.addFirst("zero");    // insert at front -- O(1)
9  list.addLast("third");   // insert at back  -- O(1)
10 String front  = list.peekFirst(); // look at front -- no remove
11 String popped = list.removeFirst(); // remove from front -- O(1)
```

Each element is a node with **prev** and **next** pointers. Insert/remove at ends: **O(1)**. Random access (`get(i)`): **O(n)**.

ArrayList vs. LinkedList: Performance Intuition

Operation	ArrayList	LinkedList
<code>get(i)</code> -- random access	$O(1)$ ✓	$O(n)$
<code>add()</code> -- append at end	$O(1)$ amortized	$O(1)$
<code>add(0, e)</code> -- insert at front	$O(n)$ (shift all)	$O(1)$ ✓
<code>remove(i)</code> -- remove by index	$O(n)$ (shift)	$O(n)$ (find first)
<code>removeFirst()</code>	$O(n)$ (shift)	$O(1)$ ✓
Memory per element	compact	extra node overhead

Use **ArrayList** when

Frequent random access (`get(i)`).

Mostly appending at the end.

Default choice for most code.

Use **LinkedList** when

Frequent insert/remove at **front** or both ends.

Queue / deque behaviour.

You don't need random access.

Collections: compact code examples

```
1  import java.util.Collections;
2
3  ArrayList<Integer> nums = new ArrayList<>(List.of(3, 1, 4, 1, 5));
4
5  Collections.sort(nums);                // ascending
6  Collections.sort(nums, Comparator.reverseOrder()); // descending
7  Collections.shuffle(nums);             // random order
8
9  int min = Collections.min(nums);
10 int max = Collections.max(nums);
```

Collections: useful helpers

`java.util.Collections` offers static utilities for **List** and related collections.

- `sort(list)` / `sort(list, comparator)` -- ordering
- `reverse(list)` / `shuffle(list)` -- reorder existing elements
- `min(list)` / `max(list)` -- extrema
- `unmodifiableList(list)` -- read-only view for defensive API returns

Note

`sort()` needs either `Comparable<T>` on elements or an explicit `Comparator`.

Array utilities (`java.util.Arrays`)

```
1  import java.util.Arrays;
2
3  int[] xs = {5, 1, 4, 2};
4  Arrays.sort(xs);                // [1, 2, 4, 5]
5  int i = Arrays.binarySearch(xs, 4); // 2 (requires sorted array)
6
7  String[] names = {"Bob", "Alice"};
8  Arrays.sort(names);             // ["Alice", "Bob"]
9  String text = Arrays.toString(names); // "[Alice, Bob]"
```

Rule of thumb

Use **Arrays** helpers for raw arrays, **Collections** helpers for **List**/collection objects.

List.of() and Immutable Lists

```
1  // Create an immutable list immediately:
2  List<String> colors = List.of("red", "green", "blue");
3
4  // Cannot modify:
5  colors.add("yellow");    // UnsupportedOperationException
6  colors.set(0, "black");  // UnsupportedOperationException
7  colors.remove(0);        // UnsupportedOperationException
8
9  // Read operations are fine:
10 String c = colors.get(1);    // "green"
11 boolean has = colors.contains("red"); // true
12 int size = colors.size();    // 3
```

When to use List.of()

Configuration constants, test data, returning a fixed set of values from a method. Use **new ArrayList<>(List.of(...))** when you need to modify the list later.

Interactive Break: Fix the Iterator

This code has a bug. Find it and fix it.

```
1  public void removeHeavyItems(double maxWeight) {  
2      for (Item item : items) {  
3          if (item.getWeight() > maxWeight) {  
4              items.remove(item);  
5          }  
6      }  
7  }
```

What exception will be thrown? Rewrite the method correctly.

Interactive Break: Fix the Iterator -- Answer

Bug: `ConcurrentModificationException`

The enhanced for-loop uses an iterator internally. Calling `items.remove()` while iterating invalidates the iterator and throws `ConcurrentModificationException`.

Fix: use `Iterator.remove()`

```
1  public void removeHeavyItems(double maxWeight) {
2      Iterator<Item> it = items.iterator();
3      while (it.hasNext()) {
4          Item item = it.next();
5          if (item.getWeight() > maxWeight) {
6              it.remove();    // safe removal through the iterator
7          }
8      }
9  }
```

Interactive Break: Fix the Iterator -- Better

(One) Answer)

Preferred modern approach (Java 8+)

```
items.removeIf(item -> item.getWeight() > maxWeight);
```

One line, reads like English -- this is the style we will use going forward.

Collection family: **Set**<T>

Purpose: model membership/uniqueness questions -- “is this already present?” -- without storing duplicates.

- **HashSet**<T>: fast membership checks, no guaranteed iteration order
- **TreeSet**<T>: keeps elements sorted, requires **Comparable** or **Comparator**
- Typical use: tags, IDs, already seen tracking, de-duplication
- Think: “Have I seen this before?” / “Keep only unique values”

Collection family: `Queue<T>`

Purpose: model processing order over time -- first-in-first-out, double-ended, or priority-driven work.

- **`ArrayDeque<T>`**: efficient FIFO queue and deque operations
- **`PriorityQueue<T>`**: serves smallest (or highest-priority) element first
- Typical use: BFS, task scheduling, next best processing
- Think: “Who gets processed next?”

Collection family: `Map<K, V>`

Purpose: model associations -- “given this key, what value belongs to it?”

- `HashMap<K, V>`: expected $O(1)$ key lookup, no guaranteed order
- `TreeMap<K, V>`: sorted keys with tree-based lookup
- `LinkedHashMap<K, V>`: preserves insertion order during iteration
- Think: lookup tables (name $\rightarrow$ object, id $\rightarrow$ record, setting $\rightarrow$ value)

Recap -- Collections Framework

- **Arrays are limited**: fixed size, manual shifting -- Collections solve this
- **Hierarchy**: `Iterable` → `Collection` → `List` → `ArrayList` / `LinkedList`
- **ArrayList**: resizable array, $O(1)$ access, $O(1)$ append -- **default choice**
- **LinkedList**: doubly-linked nodes, $O(1)$ front/back operations, $O(n)$ access
- **Enhanced for-loop**: clean iteration over any `Iterable`
- **Iterator**: safe removal while iterating -- avoids `ConcurrentModificationException`
- **Collections**: utility class with `sort`, `shuffle`, `reverse`, `min`, `max`
- `List.of()`: immutable list factory -- cannot add/remove/set

Any questions before we look at how to sort these collections?

Ordering Collections

`Comparable<T>` -- natural order

The class **defines its own** comparison logic.
“An Item knows how it compares to other Items.” Used by `Collections.sort()` without a comparator.

`Comparator<T>` -- custom order

An **external** object defines comparison.
“Someone else decides how to order Items.”
Used when natural order is not what you need.

```
1 // Works if Item implements Comparable<Item>:  
2 Collections.sort(items);  
3  
4 // Works any time -- no Comparable required:  
5 Collections.sort(items, Comparator.comparing(Item::getName));
```

Rule of thumb: implement **Comparable** for the **one obvious** natural order (e.g. ascending value). Use **Comparator** for everything else.

Game Inventory: The **Item** Class

Second running example: a simple value object for a game inventory:

```
1      public class Item {
2          private String name;
3          private double weight;    // in kg
4          private int    value;     // gold coins
5
6          public Item(String name, double weight, int value) { ... }
7          public String getName()   { ... }
8          public double getWeight() { ... }
9          public int    getValue()  { ... }
10         public String toString() { ... }
11     }
12
```

Three fields: **name**, **weight**, **value**. All getters match the field names --

Comparator.comparing(Item::getName) etc. will work directly.

Game Inventory: The **Inventory** Class

An **ArrayList**-backed container with typed operations:

```
1      public class Inventory {
2          private List<Item> items = new ArrayList<>();
3          public void addItem(Item item) { items.add(item); }
4          public int size() { return items.size(); }
5
6          public boolean removeByName(String name) {
7              Iterator<Item> it = items.iterator();
8              while (it.hasNext()) {
9                  if (it.next().getName().equals(name)) {
10                      it.remove();
11                      return true;    // found and removed
12                  }
13              }
14              return false;          // not found
15          }
16      }
```

Implementing Comparable<Item>

Let's make **Item** naturally sortable by value (lowest value first -- the conventional ascending order):

```
1  public class Item implements Comparable<Item> {
2      private String name;
3      private double weight;
4      private int value;
5
6      // ... constructor, getters ...
7
8      @Override
9      public int compareTo(Item other) {
10         // Natural order: ascending by value (lowest first)
11         return Integer.compare(this.value, other.value);
12     }
13 }
```

The `compareTo()` Contract

Your `compareTo()` implementation must follow three plain rules:

1. **Return value:** negative if `this < other`, zero if equal, positive if `this > other`.
`"a".compareTo("b") < 0   "a".compareTo("a") == 0   "b".compareTo("a") > 0`
2. **If A beats B, then B must lose to A.**
(Antisymmetry -- sorting algorithms depend on this to terminate correctly.)
3. **If A beats B and B beats C, then A must beat C.**
(Transitivity -- lets the sort confidently skip comparisons it already knows.)
4. **Bonus rule:** if `compareTo()==0`, ideally `equals()==true`.
(Consistency -- required for `TreeSet` and `PriorityQueue` to work correctly.)

Common mistake

Do **not** return `this.value - other.value` for integer comparison -- integer overflow can produce wrong results. Use `Integer.compare(this.value, other.value)` instead.

Custom Ordering: `Comparator<T>`

What if you need multiple sort orders? Use a `Comparator`:

```
1  // static final = class-level constant (same for all instances):
2  // Reusable Comparator as a named constant:
3  public static final Comparator<Item> BY_WEIGHT =
4      Comparator.comparingDouble(Item::getWeight);
5
6  public static final Comparator<Item> BY_NAME =
7      Comparator.comparing(Item::getName);
8
9  // Use them:
10 Collections.sort(items, BY_WEIGHT);           // lightest first
11 Collections.sort(items, BY_NAME);             // alphabetical
12 Collections.sort(items, BY_WEIGHT.reversed()); // heaviest first
```

Comparator: inline lambda

```
1 // Ad-hoc inline comparator (one-off sort):  
2 items.sort((a, b) -> Integer.compare(a.getValue(), b.getValue()));
```

Natural order vs. custom comparator

Comparable: “the class decides its natural order” (one order, baked in).

Comparator: “the caller decides the sort order” (many orderings, external).

Lambda Comparators: Chaining

Comparator has powerful chaining methods:

```
1 // Sort by value (descending), break ties by name (ascending):
2 Comparator<Item> byValueThenName =
3     Comparator.comparingInt(Item::getValue)
4                 .reversed()
5                 .thenComparing(Item::getName);
6
7 items.sort(byValueThenName);
```

- `Comparator.comparing(keyExtractor)`: sort by a String/Comparable key
- `comparingInt`, `comparingDouble`: use primitive-returning getters
- `.reversed()`: flip the order
- `.thenComparing(...)`: break ties with a secondary comparator

Sorting the Shape List

Putting comparators to work on the shapes running example:

```
1  ArrayList<Shape> shapes = new ArrayList<>();
2  shapes.add(new Circle(100, 100, 50, Color.RED));
3  shapes.add(new Rectangle(200, 200, 80, 40, Color.BLUE));
4  shapes.add(new Circle(50, 50, 20, Color.GREEN));
5
6  // Sort by largest area, tie-break by color name:
7  shapes.sort(
8      Comparator.comparingDouble(Shape::getArea).reversed()
9                  .thenComparing(s -> s.getColor().toString())
10 );
11
12 // Render in sorted order:
13 for (Shape s : shapes) {
14     s.draw(gc);
15 }
```

Discussion: Sorting and data structures

A **Comparator** (or **Comparable**) answers: “which of two elements should come first?” But **where** that ordering matters and **how much** of it a structure maintains depends on the **data structure**.

- **Linear data structures (lists, stacks, queues)**: You sorted a **List** of shapes; the container is a **sequence**. Order is whatever you *assign* by calling **sort**. A **stack** or **queue** defines *which* element leaves next (LIFO/FIFO) not a global key ordering. How does that **intent** differ from sorting by a key?
- **Complex data structures (trees & maps)**: For a **BST**, **inorder** traversal visits keys in sorted order **without** running **sort** on a list. Where does the ordering “live” compared to sorting an **ArrayList**?

Short turn to your neighbour: pick one bullet and discuss.

Sorting and data structures: key ideas

- **Sort on a list**: Reordering is a **batch operation** on a linear structure (e.g. TimSort). The list does not *enforce* sorted order between calls you re-sort when needed.
- **Stack / queue**: These specify an **access policy** (LIFO/FIFO), not “all elements ordered by key” a different contract than **Comparator** ordering.
- **BST / sorted maps**: **Comparison** is built into **insert** and **search**; inorder traversal **reads out** keys in order the tree **encodes** the ordering.

Recap -- Sorting

- **Comparable<T>**: “natural order” -- the class defines how it compares to itself
- **Comparator<T>**: external order -- caller decides how to sort
- **compareTo()** contract: antisymmetry, transitivity, consistency with **equals**
- **Never** use subtraction for integer comparison -- use **Integer.compare()** instead
- **Comparator.comparing()** / **comparingInt** / **comparingDouble**: lambda-friendly factories
- **.reversed()** and **.thenComparing()**: chain comparators for multi-key sorting
- **TreeSet** and **PriorityQueue** use the same **Comparable/Comparator** contracts (see Collection family slides earlier)

Any questions before we put everything together?

Inventory (1/4): generics & simple API

Encapsulate a `List<Item>`; callers work with **Inventory**, not the raw list.

```
1  public class Inventory {
2      private List<Item> items = new ArrayList<>();
3      public void addItem(Item item) {
4          items.add(item);
5      }
6      public int size() {
7          return items.size();
8      }
9      @Override
10     public String toString() {
11         return items.toString();    // delegates to Item.toString()
12     }
13     // ...
14 }
```

-- `List<Item>`: one concrete type parameter everywhere compile-time safety

-- `toString`: list delegates to each element; useful for debugging and demos

Inventory (2/4): remove by name with **Iterator**

We need **whether** a match existed not just “remove all matches”. **removeIf** does not tell you if anything was removed in one boolean.

```
1      public boolean removeByName(String name) {
2          Iterator<Item> it = items.iterator();
3          while (it.hasNext()) {
4              if (it.next().getName().equals(name)) {
5                  it.remove();
6                  return true;
7              }
8          }
9          return false;
10     }
```

*First match only; safe removal via **Iterator.remove()** (same pattern as earlier in this lecture).*

Inventory (3/4): two ways to sort

Natural order (Item implements Comparable<Item>) vs. **caller-defined** order (Comparator).

```
1      public void sortByValue() {  
2          Collections.sort(items);  
3      }  
4  
5      public void sortBy(Comparator<Item> comparator) {  
6          items.sort(comparator);  
7      }
```

Collections.sort VS. list.sort

Both are fine for a **List**: **Collections.sort(items)** and **items.sort(comparator)** use whichever reads clearer in context.

Inventory (4/4): full class

```
1  public class Inventory {
2      private List<Item> items = new ArrayList<>();
3      public void addItem(Item item) { items.add(item); }
4      public int size() { return items.size(); }
5      public boolean removeByName(String name) {
6          Iterator<Item> it = items.iterator();
7          while (it.hasNext()) {
8              if (it.next().getName().equals(name)) {
9                  it.remove(); return true;
10             }
11         }
12         return false;
13     }
14     public void sortByValue() { Collections.sort(items); }
15     public void sortBy(Comparator<Item> comparator) {
16         items.sort(comparator);
17     }
18     @Override
19     public String toString() {
20         return items.toString();
21     }
22 }
```

Optional<T>: when a result might be missing

Problem: A search (by name, id, ...) sometimes finds **nothing**. The API must say so but *how* shapes how callers write code.

- **null**: “no value” is a valid reference. Easy to return, easy for callers to **forget** to check **NullPointerException** at runtime.
- **Optional<T>**: A small wrapper meaning “either **one T** or **no** value”. Not a collection at most one element. The type appears in the method signature, so “might be absent” is **visible** at compile time.

Factory methods

Optional.of(x) value present; **Optional.empty()** absent.

Finding Items: Linear Search

Two styles for “maybe not found”: return **null**, or return **Optional<Item>**.

```
1  public Item findByName(String name) {
2      for (Item item : items) {
3          if (item.getName().equals(name)) return item;
4      }
5      return null;    // not found
6  }
7
8  public Optional<Item> findByNameSafe(String name) {
9      for (Item item : items) {
10         if (item.getName().equals(name)) return Optional.of(item);
11     }
12     return Optional.empty();
13 }
```

Why **Optional<T>**?

Returning **Optional** forces the caller to handle the “not found” case explicitly. Returning **null** is easy to forget -- and causes a **NullPointerException** later.

Finding Items: using **Optional** (caller side)

Once the method returns **Optional<Item>**, the caller chooses how to react:

```
1  Optional<Item> result = inv.findByNameSafe("Sword");
2
3  // ifPresent: runs the lambda only if a value is present
4  result.ifPresent(item -> System.out.println("Found: " + item));
5
6  // isPresent / get: explicit check before access
7  if (result.isPresent()) {
8      System.out.println("Value: " + result.get().getValue());
9  }
```

- **ifPresent**: run a side effect when there is a value no **get()**, often the clearest style
- **isPresent()** / **get()**: use when you need branches; prefer **orElse** / **orElseThrow** in real code over bare **get()**

Predicate<T>: a yes/no test on one value

Idea: A **predicate** answers “does this **T** satisfy a condition?” one input, **boolean** out.

- `java.util.function.Predicate<T>` is a **functional interface**: a single abstract method `boolean test(T t)`.
- You supply a **lambda** (or method reference) wherever a `Predicate<T>` is expected; the body must match `test` (argument **T**, result **boolean**).
- Same package also defines **Consumer**, **Function**, **Supplier**, ... each is a small “shape” for a lambda.

```
1  import java.util.function.Predicate;
2
3  Predicate<String> nonEmpty = s -> !s.isEmpty();
4  nonEmpty.test("hello");    // true
5
6  // Any condition on Item can be a Predicate<Item>:
7  Predicate<Item> light = item -> item.getWeight() < 1.0;
```

findFirst with Predicate (1/2): definition

Same linear scan as `findByName`, but the **condition** is a parameter of type `Predicate<T>` reusable for any element type and any test.

```
1  import java.util.function.Predicate;
2
3  // Generic utility -- works for any list and any condition:
4  public static <T> Optional<T> findFirst(List<T> list,
5                                         Predicate<T> condition) {
6      for (T element : list) {
7          if (condition.test(element)) return Optional.of(element);
8      }
9      return Optional.empty();
10 }
```


findFirst with Predicate (2/2): usage

```
1 // Find by name:
2 Optional<Item> sword = findFirst(items,
3     item -> item.getName().equals("Sword"));
4
5 // Find lightest item:
6 Optional<Item> lightest = findFirst(items,
7     item -> item.getWeight() < 1.0);
8
9 // Works on shapes too:
10 Optional<Shape> bigShape = findFirst(shapes,
11     s -> s.getArea() > 1000.0);
```

Same **Predicate** idea---library pipelines

Stream's **filter** method takes a **Predicate**. You can express the same conditions as in **findFirst**, but compose them with **map**, **sorted**, and **collect**.

Java Streams: motivation

- **External iteration:** you write the `for` loop or `Iterator` and *pull* each element.
- **Internal iteration:** you describe *what* you want---`filter`, `map`, `sort`---and the library applies it while walking the data.
- Streams reuse the **same functional interfaces**: `Predicate`, `Function`, `Comparator`.

A stream is not required for every loop---plain loops stay fine. Reach for streams when a short pipeline reads clearer than nested loops.

Creating a stream

Every **Collection** exposes **stream()** (and **parallelStream()**, not our focus today). You can also build tiny streams directly:

```
1  import java.util.stream.Stream;
2
3  List<Item> items = new ArrayList<>();
4  // ...
5  Stream<Item> fromList = items.stream();
6  Stream<String> letters = Stream.of("a", "b", "c");
```

Streams: pipeline shape

- **Source**: e.g. `list.stream()` or `Stream.of(...)`.
- **Intermediate operations**: `filter`, `map`, `sorted`, ...---each returns another **Stream**; evaluation is **lazy**.
- **Terminal operation**: `forEach`, `collect`, `findFirst`, `count`, ...---**triggers** work and yields a result or effect.

Until a terminal runs, intermediate steps only describe the pipeline; the library may fuse passes for efficiency.

filter---Predicate again

`filter(Predicate<T>)` keeps elements where **test** is true---the same boolean condition you passed to `findFirst`.

```
1  import java.util.stream.Collectors;
2
3  List<Item> items = new ArrayList<>();
4  // ...
5
6  List<Item> heavy = items.stream()
7      .filter(item -> item.getWeight() > 5.0)
8      .collect(Collectors.toList());
```

map--Function<T,R>

map applies a one-argument transformation to each element. The lambda matches **Function<T,R>** (input type **T**, result type **R**).

```
1 List<String> names = items.stream()
2   .map(Item::getName)
3   .collect(Collectors.toList());
```

sorted in a pipeline

Use natural order if **T** is **Comparable**, or pass a **Comparator**---same factories as **List.sort** (**Comparator.comparing...**).

```
1  List<Item> byValue = items.stream()
2      .sorted(Comparator.comparingInt(Item::getValue))
3      .collect(Collectors.toList());
4
5  List<Item> byWeightDesc = items.stream()
6      .sorted(Comparator.comparingDouble(Item::getWeight).reversed())
7      .collect(Collectors.toList());
```

collect: turn a stream back into a collection

A **Stream**<T> is **not** a **List** or **Set**---it is a one-shot pipeline. **collect** is the usual terminal when you want a concrete **collection** (or string) at the end.

- You pass a **Collector** describing *how* to gather elements (list, set, concatenated string, ...).
- Static factories live in **Collectors**: **toList()**, **toSet()**, **joining(...)**, and many more.

```
1 List<String> names = items.stream()
2   .map(Item::getName)
3   .collect(Collectors.toList()); // new ArrayList-like list (implementation may
4   vary)
5 String joined = items.stream()
6   .map(Item::getName)
7   .collect(Collectors.joining(", ")); // one String
```

Until you **collect** (or **reduce** / **sum** / ...), you only have a description of steps---no new **List** exists yet.

Common terminal operations

```
1    long n = items.stream()
2        .filter(item -> item.getValue() > 50)
3        .count();
4
5    Optional<Item> sword = items.stream()
6        .filter(item -> item.getName().equals("Sword"))
7        .findFirst();
8
9    items.stream()
10        .map(Item::getName)
11        .forEach(name -> System.out.println(name));
```

findFirst() yields an **Optional**---the same “maybe nothing” story as **findByNameSafe**.

Worked example: legendary item names

High-value items (value > **100**), lightest first, then collect their names:

```
1  import java.util.stream.Collectors;
2
3  List<String> legendNames = items.stream()
4      .filter(item -> item.getValue() > 100)
5      .sorted(Comparator.comparingDouble(Item::getWeight))
6      .map(Item::getName)
7      .collect(Collectors.toList());
```

Streams: pitfalls

- A **Stream** is **single-use**: after a terminal, it is consumed---call `collection.stream()` again for another pass.
- Prefer **collect** / **reduce** over complicated side effects in **forEach**; easier to test and parallelize later.
- **parallelStream()** can use multiple cores but needs correct, side-effect-free operations---treat as **advanced**; stick to sequential streams until you study concurrency.

Quick Retrieval: Test Yourself

Before looking at the summary -- can you answer these from memory?

1. What problem does `Box<T>` solve compared to `Box` with an `Object` field?
2. What does `<T extends Shape>` mean, and why is the bound needed?
3. When should you use `LinkedList` instead of `ArrayList`?
4. You write `for (Item i : items) { items.remove(i); }` -- what happens and how do you fix it?
5. What is the difference between `Comparable<T>` and `Comparator<T>`?

Take some time to think, then we go through the answers.

Quick Retrieval: Answers

1. **Type safety:** `Box<String>` lets the compiler enforce that only Strings go in/out. `Object` requires casts and allows `ClassCastException` at runtime.
2. **Bounded type parameter:** `T` must be `Shape` or a subtype. Needed to call `Shape` methods (e.g. `getArea()`) on a value of type `T` -- without the bound, `T` is just `Object`.
3. **LinkedList:** when you frequently insert/remove at the **front** or both ends ($O(1)$ vs. $O(n)$). `ArrayList` is better for indexed access and appending.
4. **ConcurrentModificationException.** Fix: use `Iterator<Item> it = items.iterator(); while(it.hasNext()) { if(...) it.remove(); }`
5. **Comparable:** the class defines its own natural order (`compareTo` inside the class).
Comparator: an external object defines a specific ordering -- useful for multiple sort orders or types you don't control.

Summary

Generics

- `<T>`: type-safe, reusable code
- Errors at **compile time**, not runtime
- `<T extends X>`: bounded type
- `<>`: diamond operator

Collections

- **ArrayList**: default choice, $O(1)$ access
- **LinkedList**: $O(1)$ front/back
- **Iterator** / **removeIf()**: safe removal
- **Collections**, **List.of()**: utilities

Sorting

- **Comparable**: natural order (inside class)
- **Comparator**: external order (lambda)
- **.reversed()**, **.thenComparing()**: chaining

Streams

- **Pipeline**: source; intermediates (**filter**, **map**, **sorted**); terminal (**collect**, **findFirst**, ...)
- **Lazy** evaluation.
- **Single-use** stream; prefer **collect** over heavy side effects in **forEach**

Backup: Map.Entry<K,V> iteration patterns

```
1  Map<String, Integer> scores = new HashMap<>();
2  scores.put("Alice", 95); scores.put("Bob", 87); scores.put("Carol", 92);
3  // Pattern 1: iterate keys only
4  for (String name : scores.keySet()) {
5      System.out.println(name);
6  }
7  // Pattern 2: iterate values only
8  for (int score : scores.values()) {
9      System.out.println(score);
10 }
11 // Pattern 3: iterate key-value pairs (most common)
12 for (Map.Entry<String, Integer> entry : scores.entrySet()) {
13     System.out.println(entry.getKey() + " -> " + entry.getValue());
14 }
15 // Pattern 4: Java 8+ forEach with lambda
16 scores.forEach((name, score) ->
17     System.out.println(name + ": " + score));
```

Backup: Set<T> and Uniqueness

A **Set** contains **no duplicate elements**:

```
1 // HashSet: fast (O(1)) but no guaranteed order
2 HashSet<String> tags = new HashSet<>();
3 tags.add("magic");
4 tags.add("fire");
5 tags.add("magic"); // duplicate -- silently ignored
6 System.out.println(tags.size()); // 2
7
8 // TreeSet: sorted order (requires Comparable or Comparator)
9 TreeSet<String> sortedTags = new TreeSet<>(tags);
10 // iteration in alphabetical order: "fire", "magic"
11
12 // Useful operations:
13 boolean added = tags.add("ice"); // returns false if already present
14 tags.contains("fire"); // O(1) for HashSet
```

When you need to guarantee uniqueness and don't care about order (**HashSet**), or want sorted unique elements (**TreeSet**).

Backup: Why Generic Arrays Are Tricky

Drawing<T> in the slides uses this pattern:

```
1    @SuppressWarnings("unchecked")
2    shapes = (T[]) new Object[capacity];
```

Why can't we write `new T[capacity]` directly? Because Java erases **T** at runtime -- at the bytecode level, **T** becomes **Object**. The JVM would not know what type of array to allocate. **new T[capacity]** is therefore a compile error.

The workaround creates an **Object []** and casts it. The **@SuppressWarnings** suppresses the unchecked cast warning because we (the developer) know the array will only ever hold **T** objects.

In practice: Use **ArrayList<T>** internally -- it handles this internally and is the idiomatic Java solution. The array trick only appears when building low-level data structures for educational or performance reasons.

Backup: Defensive Copies `unmodifiableList`

Returning internal collections directly exposes your object's state:

```
1  public class Inventory {
2      private ArrayList<Item> items = new ArrayList<>();
3      // DANGEROUS: caller can modify your private list!
4      public ArrayList<Item> getItems() { return items; }
5
6      // SAFE option 1: return an unmodifiable view
7      public List<Item> getItems() {
8          return Collections.unmodifiableList(items);
9      }
10     // SAFE option 2: return a defensive copy
11     public List<Item> getItemsCopy() {
12         return new ArrayList<>(items);
13     }
14 }
```

Which to use? `unmodifiableList`: caller can read, not write -- cheaper (no copy). Defensive copy: caller gets their own list but changes don't affect you.